

A Resident State Machine for Deterministic Context Execution in Large Language Models

Author: Shannon Birchell **Role:** Researcher / Author **Date:** July 14, 2026

Abstract

Large language models are powerful autoregressive systems, but their execution across multiple interactions can be difficult to constrain, inspect, and reproduce. This paper proposes a resident state machine: a persistent state machine represented directly inside the model context. The state machine defines the active execution state, the valid transitions from that state, and the state-specific context that should be loaded for the next interaction.

The central question is:

Can a persistent state machine embedded in the context improve structural determinism, traceability, and task execution by constraining an LLM to execute exactly one state per interaction?

This paper intentionally focuses on a single resident state machine. It does not attempt to solve multi-agent orchestration, agent operating systems, persistent memory architectures, planning systems, or context compression.

Resident State Machine Execution Loop

Execution state is resident in the same context window used for generation.

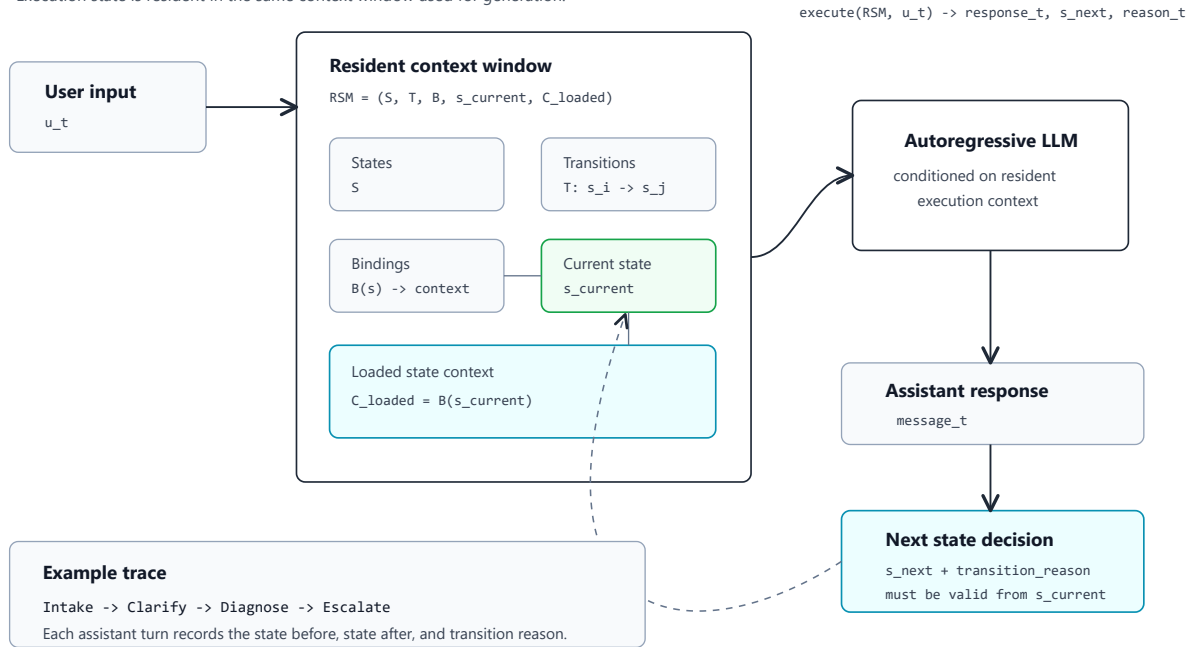


Figure 1. Resident State Machine Execution Loop. The current state, valid transitions, transition reasoning, and loaded state context remain resident in the same context window used for generation. The model produces an assistant response and a next-state decision, which updates the resident state for the following interaction.

1. Problem Statement

LLM applications often need to maintain continuity across turns. In practice, this continuity is frequently managed through external orchestration code, hidden application state, long prompts, tool-routing frameworks, or informal conversational memory.

These approaches can work, but they often make execution difficult to inspect. A reviewer may see the user prompt and model response, but not the execution controller that shaped the response. This limits traceability and makes it harder to determine why the model behaved as it did.

A resident state machine addresses this by placing the execution controller inside the context that the model reads. The machine is not hidden orchestration. It is part of the prompt substrate.

2. Core Claim

The contribution is not that state machines exist. The contribution is representing the execution state machine itself as resident context, allowing an autoregressive LLM to function as a constrained state executor with deterministic transition boundaries across multiple interactions.

In this model, the LLM is not asked to freely infer the entire task structure at each turn. Instead, it is given:

1. The full set of states.
2. The current state.
3. The valid transitions from the current state.
4. The reasoning criteria for choosing each transition.
5. The context bound to the current state.
6. A requirement to execute exactly one state decision per interaction.

3. Resident State Machine Definition

A resident state machine is a state machine serialized into the prompt context and preserved across interactions.

Formally, a resident state machine can be represented as:

```
RSM = (S, T, B, s_current, C_loaded)
```

Where:

- `S` is a finite set of named states.
- `T` is a finite set of valid transitions, where each transition is a tuple `(s_from, s_to, reason)`.
- `B` is a context-binding function that maps each state to state-specific instructions or context: `B(s) -> context`.
- `s_current` is the active state at the start of an interaction.
- `C_loaded` is the active context inserted into the prompt for the current state, where `C_loaded = B(s_current)`.

For a single interaction, the model receives the machine topology, the current state, valid transition reasoning, the loaded state context, and the most recent user input. It returns an assistant response plus a proposed next state:

```
execute(RSM, user_input) -> (assistant_response, s_next, transition_reason)
```

The next state must satisfy:

```
s_next = s_current  
or  
(s_current, s_next, reason) in T
```

The option to remain in `s_current` allows the system to avoid premature transition when the active state still requires more evidence.

At minimum, it contains:

```
context_resident_state_machine {
  machine: Customer Support Triage
  states: Intake, Clarify, Diagnose, Resolve, Escalate, Close
  associations: Intake -> Clarify, Intake -> Diagnose, Clarify -> Diagnose
  current_state: Clarify
}

transition_reasoning {
  Intake -> Clarify: the issue lacks required details
  Intake -> Diagnose: the issue has enough detail to infer likely causes
  Clarify -> Diagnose: the missing detail has been supplied
}

state_context_bindings {
  Intake: context bound
  Clarify: context bound
  Diagnose: context bound
}

loaded_state_context {
  state: Clarify
  context:
    Ask targeted follow-up questions needed to diagnose the issue.
}
```

The complete transition topology remains visible, but only the selected state's task instructions are loaded as active execution context.

4. Execution Rule

The model is constrained to execute exactly one state per interaction.

For each user turn, the model should:

1. Read the resident state machine.
2. Read the current state.
3. Use only the loaded state context to generate the assistant response.
4. Evaluate the most recent user turn against the valid transitions.

5. Select the next state from the allowed transition set, or remain in the current state.
6. Emit the state decision in a structured form.

An example structured state decision is:

```
{
  "assistantMessage": "[assistant response]",
  "nextStateId": 2,
  "nextStateName": "Clarify",
  "stateReason": "the issue lacks required details such as product area, error message,
account, or urgency"
}
```

5. Why Resident Execution Fits Autoregressive Transformers

Autoregressive transformers generate each next token conditioned on the preceding context. A resident state machine uses this property directly: the execution controller is placed inside the conditioning context rather than hidden outside the model. The model's next-token distribution is therefore conditioned not only on the conversation, but also on an explicit current state, a finite transition set, and state-specific execution instructions.

This alignment matters because the model does not need to reconstruct the task phase solely from conversational history. The active state and valid next states are present in the same context window used for generation. The resident machine acts as a local execution frame that biases the model toward one state-specific behavior and one bounded state decision.

The model still generates language probabilistically. Resident execution does not make token output deterministic. Instead, it gives the model a stable computational scaffold:

- the current state identifies the active task mode;
- the loaded state context supplies local instructions;
- transition reasoning narrows the next-state decision;
- the emitted state decision makes the execution path inspectable.

In this sense, resident execution fits autoregressive behavior because it turns state into conditioning context. The machine is not an external controller issuing invisible commands; it is part of the sequence the model attends to when producing the next response.

6. Determinism

The resident state machine improves structural determinism by narrowing the model's execution choices. The model does not need to choose from an unbounded set of possible task modes. It chooses from the current state's valid transitions.

This does not make the model deterministic in the mathematical sense, nor does it demonstrate deterministic natural-language output. Sampling, model updates, and language ambiguity can still affect response text. However, it introduces deterministic execution framing around the model's path:

- The current state is explicit.
- The allowed transitions are explicit.
- The reasoning for each transition is explicit.
- The selected next state is recorded.

The resulting execution trace can be inspected after the fact.

7. Traceability

Traceability is one of the strongest practical benefits of the resident state machine.

Each interaction can store:

- User message.
- Assistant message.
- State before response.
- State after response.
- Transition reason.
- Raw model output.
- Loaded state context.

This makes it possible to analyze not only what the model said, but which state it believed it was executing and why it selected the next state.

For example:

```
Assistant - State: Clarify
State decision: Clarify -> Diagnose
Reason: User supplied product, account, error text, browser, start time, and scope.
```

This creates a lightweight execution audit trail.

8. Task Execution

The resident state machine is useful when the task has recognizable phases. A customer support triage workflow might include:

```
Intake -> Clarify -> Diagnose -> Resolve -> Close
Diagnose -> Escalate
Resolve -> Escalate
Escalate -> Close
```

The workflow remains explicitly represented throughout execution, reducing reliance on implicit reconstruction from conversational history. The model's job is to execute the active state and choose a valid next state.

9. Experimental Prototype

The prototype application implements a simple resident state machine builder and testing harness.

It supports:

- Creating named machines.
- Creating named states.
- Binding context to each state.
- Defining valid state associations.
- Providing reasoning for each valid transition.
- Rendering the resident state machine as LLM context.
- Running a live conversation against the machine.
- Saving test transcripts for analysis.

9.1 Prototype Architecture

The prototype separates machine design from machine testing. The browser builds the resident context from the selected machine: state names, associations, transition reasoning, current state, and the loaded state context. The backend then wraps that resident context with the recent conversation transcript, the most recent user message, the valid next states, and the transition reasons from the current state before calling the model.

This means the resident machine is visible to the model, but the prototype also supplies a narrow decision frame for the current turn.

9.2 Prototype Enforcement Model

The prototype uses two enforcement layers.

First, the model is prompted with only the valid next states from the current state, plus the option to remain in the current state. The current-state option is intentionally included even when it is not represented as an explicit association in the machine definition. This allows the model to stay in a state when evidence is incomplete.

Second, the backend validates the returned state decision before applying it. If the model emits a state that is not in the valid next-state set, the backend falls back to the current state. The raw model decision is still stored for analysis, but the applied state remains within the machine boundary.

This distinction is important: the resident state machine is the conceptual controller, while the prototype implementation adds guardrails around model output.

9.3 Experimental Method

The current experiment uses hand-authored demo machines and OpenAI-backed live conversation runs. Each run is saved to SQLite as a test session with user messages, assistant messages, state-before, state-after, transition reason, model name, and raw model output. The current fixture exports those machines and transcripts into `fixtures/demo_state_machines.json` so the experiment state can be loaded and inspected by future users.

The analysis is qualitative. It examines observed transition paths and stored transition reasons rather than claiming statistical performance.

In a customer support triage test, a vague login issue began in `Intake`, moved to `Clarify`, then to `Diagnose`, and then to `Escalate` when customer-provided details indicated an account-specific SSO issue requiring privileged investigation.

This illustrates the core hypothesis: the LLM can follow a resident state machine, produce useful task responses, and emit traceable state transitions.

10. Transcript Analysis

The prototype currently includes four saved test sessions across three machines. These transcripts are not a statistical evaluation, but they provide early qualitative evidence about whether the resident state machine can constrain execution and produce traceable state transitions.

The saved sessions contain:

```
3 machines
4 test sessions
24 transcript messages
12 assistant turns
```

The fixture composition is:

```
Customer Support Triage: 2 sessions, 10 messages
Product Return Authorization: 1 session, 6 messages
Event Incident Response: 1 session, 8 messages
```

The observed state paths were:

Customer Support Triage smoke test: Resolve -> Close
Customer Support Triage: Intake -> Clarify -> Diagnose -> Escalate -> Escalate
Product Return Authorization: Receive -> Validate -> Assess -> Assess
Event Incident Response: Report -> Clarify -> Classify -> Respond -> Document

Machine	Session Type	Observed Path	Notable Behavior	Evidence Source
Customer Support Triage	Transcript persistence smoke test	Resolve -> Close	Confirmed saved transcript path and close behavior for a short response	fixtures/demo_state_machines.json
Customer Support Triage	Support triage scenario	Intake -> Clarify -> Diagnose -> Escalate -> Escalate	Progressed through missing-details, diagnosis, escalation, then retained escalation while awaiting handoff artifacts	fixtures/demo_state_machines.json
Product Return Authorization	Return request scenario	Receive -> Validate -> Assess -> Assess	Advanced to assessment, then retained assessment while awaiting defect evidence	fixtures/demo_state_machines.json
Event Incident Response	Event incident scenario	Report -> Clarify -> Classify -> Respond -> Document	Advanced through classification and response into incident documentation	fixtures/demo_state_machines.json

10.1 Valid Transition Following

Across the transcript set, applied state changes remained within the valid transition boundary provided by the resident machine context. The model was prompted with only valid next states and the current-state retention option; the backend also validated the returned state before applying it. The assistant turns include both a response and a state decision, allowing each transition to be inspected after the interaction.

For example, the customer support login issue began with an underspecified report:

Intake -> Clarify

Reason: the issue lacks required details such as product area, error message, account, or urgency

After the user supplied product, account, error text, browser, timing, and scope, the model selected:

Clarify -> Diagnose

When the troubleshooting results indicated likely SSO/session infrastructure causes requiring privileged access, the model selected:

Diagnose -> Escalate

This path is consistent with the transition reasoning encoded in the machine.

10.2 State Retention As A Useful Outcome

Two transcripts show that remaining in the same state can be useful rather than erroneous.

In the customer support session, the model remained in `Escalate` after preparing an escalation package because it still needed concrete handoff artifacts:

Escalate -> Escalate

Reason: the escalation package is prepared but awaiting user IDs, timestamps, HAR/screenshots, and logs

In the product return session, the model remained in `Assess` because the user had provided order and timing details but had not yet provided defect evidence or troubleshooting results:

Assess -> Assess

Reason: still need defect evidence and troubleshooting results before approving the return

This suggests that a resident state machine should explicitly allow state retention when more evidence is needed. In the prototype, current-state retention is supplied as an allowed next state by the execution harness rather than modeled as a self-association in the machine topology. Without that option, the model may be forced into premature approval, denial, closure, or escalation.

10.3 Traceability Of Reasoning

The transcript structure makes the model's execution path auditable. Each assistant turn records:

- State before response.
- State after response.
- Natural-language transition reason.
- Raw model output.

This allows post-hoc analysis of both task response quality and state transition quality. Instead of asking only whether the assistant's text sounded reasonable, the reviewer can ask whether the model was executing the correct state and whether the selected transition followed the machine definition.

10.4 Early Findings

These initial transcripts support three early observations:

1. A resident state machine can keep the model oriented around a specific workflow across turns.
2. Transition reasoning provides useful audit material for explaining state movement.
3. The ability to remain in the current state is important for avoiding premature task closure.

These observations are preliminary. The transcript set is small, hand-designed, domain-specific, and generated from cooperative scenarios rather than adversarial or highly ambiguous ones. A stronger evaluation would require repeated trials, independent scoring of transition correctness, and comparison against prompts without resident state machines.

11. Explicit Non-Goals

This paper does not address:

- Multi-agent orchestration.
- BIAB integration.
- Context compression.
- Agent operating systems.
- Skills and harnesses.
- Reflection frameworks.
- Persistent memory architectures.
- Planning systems.
- Hierarchical state machines.
- Distributed execution.

These may be useful extensions, but including them here would dilute the central claim.

12. Future Work

Future papers can build outward from this foundation:

1. Resident State Machines for LLM Execution.
2. Hierarchical Resident State Machines.
3. Resident State Machines for Multi-Agent Coordination.
4. Context Kernels: A Computational Model for Agent Memory and Execution.

Each extension should preserve the core discipline of making execution state explicit, inspectable, and resident in context.

13. Conclusion

A resident state machine offers a narrow, inspectable mechanism for constraining LLM execution across interactions. By embedding the machine directly into context, the model can act as a state executor rather than a free-form conversational agent.

The key contribution is representing the execution state machine itself as resident context, allowing an autoregressive LLM to function as a constrained state executor with deterministic transition boundaries across multiple interactions.

This work intentionally limits itself to a single resident state machine. It does not address multi-agent systems, distributed execution, or hierarchical state machines. Those extensions are left for future work.